SECURITY

SECURITY REVIEW · PREPARED FOR

letscash

Airdrop vault for letscash launches on Uniswap v4, Robinhood Chain

DATE	COMMIT	REMEDIATION
24.08.2026	1ecf5859	4e190eb6

01 ABOUT / DISCLAIMER / RISK CLASSIFICATION

Phase & process

1. About SBSecurity

SBSecurity is a team of skilled smart contract security researchers. Based on the audits conducted and numerous vulnerabilities reported, we strive to provide the absolute best security service and client satisfaction. While it's understood that 100% security and bug-free code cannot be guaranteed by anyone, we are committed to giving our utmost to provide the best possible outcome for you and your product.

Book a Security Review with us at sbsecurity.net or reach out on Telegram @blckhv.

2. Disclaimer

A smart contract security review can only show the presence of vulnerabilities **but not their absence**. Audits are a time, resource, and expertise-bound effort where skilled technicians evaluate the codebase and their dependencies using various techniques to find as many flaws as possible and suggest security related improvements. We as a company stand behind our brand and the level of service that is provided but also recommend subsequent security reviews, on-chain monitoring, and high whitehat incentivization.

3. Risk classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

3.1 IMPACT

- **High** - leads to a significant loss of assets in the protocol or significantly harms a group of users.
- **Medium** - leads to a moderate loss of assets in the protocol or some disruption of the protocol's functionality.
- **Low** - funds are not at risk.

3.2 LIKELIHOOD

- **High** - almost certain to happen, easy to perform, or highly incentivized.
- **Medium** - only conditionally possible, but still relatively likely.
- **Low** - requires specific state or little-to-no incentive.

02 EXECUTIVE SUMMARY

Key findings

TOTAL FINDINGS	RESOLVED	HIGH SEVERITY FINDINGS
1	1 of 1 · 100%	0 none found

SEVERITY DISTRIBUTION

Fixed & verified Acknowledged

CRITICAL	0
HIGH	0
MEDIUM	0
LOW	1
INFORMATIONAL	0
GAS OPT.	0

CLIENT	REPOSITORY	METHODS
letscash	Private	Manual review

CONTRACTS IN SCOPE

`CashCatAirdropVault.sol`

TIMELINE · AUGUST 2026

Audit · 3 days | Remediation · 1 day

KICK-OFF	END OF AUDIT	REMEDIATIONS START	REMEDIATIONS END
19.08.2026	21.08.2026	24.08.2026	24.08.2026

STARTING COMMIT HASH	`1ecf5859a896760f118fdaf6dde845834be23cdb`
FINAL COMMIT HASH	`4e190eb6f4ab5f804b4ef20f1a86efdf73ac65f1`

03 PROJECT SUMMARY

What letscash is

CashCatAirdropVault holds the share of a launch's first buy that is set aside as an airdrop. It pays out only along a recipient list the creator publishes on chain in advance, in full, after a public countdown; whatever is still held at the deadline can be burned by anyone. There is no function that sends the token to an address of the caller's choosing.

SECURITY POSTURE

BEFORE REMEDIATION 1ECF5859

Creator hand-over

L-01

The vault's creator was written once at launch and could not follow CashCatHookV2.updateCreator, so a routine hand-over of the fee stream left the airdrop publishable only by the retired key, and the allocation would burn at the deadline for a reason invisible from chain state.

AFTER REMEDIATION 4E190EB6

The finding was resolved at the remediation commit. The vault gained transferCreator, callable by the current creator, so the publish right can follow a launch hand-over, and the hook's updateCreator documentation now directs a front-end to send both calls together. The two-call shape was kept deliberately: a v4 pool binds its hook into its identity at creation, so an atomic rebind would have required a new hook that no launched pool could use.

The remediation commit range also moves the factory's supply bound from the launch configuration to the amount that actually reaches a vault. That change is unrelated to the finding and was not reviewed.

04 RECOMMENDATIONS

Beyond this audit

Measures that extend the protection of this review after launch, independent of any individual finding.

R-01	**Send both hand-over calls from the product**	Wherever a creator can call updateCreator, send transferCreator in the same flow and warn while airdropVaultOf(poolId) holds a vault whose creator still differs, so the two never drift by accident.
R-02	**Ship a watcher for CampaignSealed**	The countdown is a public record, not a notice: nothing on chain tells a recipient they are on a list. A watcher on CampaignSealed and executableAt is what turns the record into a warning, and product copy should say record until it exists.

06 CONTRACTS & ROLES

Core contracts

CONTRACT	ROLE
`CashCatAirdropVault.sol`	Holds a launch's airdrop allocation as an EIP-1167 clone per launch. Campaigns are declared with a Merkle fingerprint, published in parts, sealed automatically, paid out in batches after the countdown, and anything left burns after the deadline.

06 CONTRACTS & ROLES

Privileged roles

Creator.

The address set at launch, and since the fix transferable by its holder. Opens a campaign, publishes its parts, and may discard an unsealed draft. Cannot move tokens anywhere a published list does not name.

Factory.

Deploys the vault clone at launch and calls initialize once, binding the token and the creator.

Anyone.

Executes batches of a sealed campaign once the countdown has elapsed, and burns whatever remains after the deadline.

07 FINDINGS

What we found

ID	TITLE	SEVERITY	STATUS
L-01	Vault creator cannot follow CashCatHookV2.updateCreator - a routine fee-stream creator change silently strands the airdrop-publish capability	LOW	FIXED

LOW SEVERITY · 1 FINDING

L-01 LOW FIXED

Vault creator cannot follow CashCatHookV2.updateCreator - a routine fee-stream creator change silently strands the airdrop-publish capability

`CashCatAirdropVault::creator` is written once, in `initialize`, and no function in the vault writes it again. Every publish path (`openCampaign`, `publishPart`, `discardDraft`) is gated by `onlyCreator`.

https://github.com/SB-Security/audit-2026-07-CashCat/blob/1ecf5859a896760f118fdaf6dde845834be23cdb/src/v4/CashCatAirdropVault.sol#L595-L613

```
 modifier onlyCreator() {
    if (msg.sender ≠ creator) revert NotCreator();
    _;
}

function initialize(address token_, address creator_) external {
    if (isMaster) revert MasterCannotBeInitialized();
    if (token ≠ address(0)) revert AlreadyInitialized();
    if (token_ == address(0) || creator_ == address(0)) revert ZeroAddress();

    token = token_;
    creator = creator_;
    ...
}
```

`CashCatHookV2::updateCreator` lets a creator hand the launch to a new address, for example a multisig. It updates only `poolConfigs[poolId].creator`. The vault is not touched.

https://github.com/SB-Security/audit-2026-07-CashCat/blob/1ecf5859a896760f118fdaf6dde845834be23cdb/src/v4/CashCatHookV2.sol#L613-L620

```
 function updateCreator(PoolId poolId, address newCreator) external {
    PoolConfig storage config = poolConfigs[poolId];
    if (!config.exists) revert UnknownPool();
    if (msg.sender ≠ config.creator) revert NotCreator();
    if (newCreator == address(0)) revert ZeroAddress();
    emit CreatorUpdated(poolId, config.creator, newCreator);
    config.creator = newCreator;
}
```

After the hand-off the pool's creator and the vault's `creator` are different addresses. The new creator, the one the product now shows, gets `NotCreator` from the vault. The old key is the only address that can publish, and there is no way to rebind it.

Both contracts mention this in comments, but nothing on chain prevents it or recovers from it.

IMPACT

L-01 · CONTINUED

- A creator who hands the stream to a multisig and retires the old key can never publish the airdrop. The allocation sits in the vault until `deadline`, then anyone burns it with `burnAfterDeadline`.
- A creator who rotates away from a compromised key does not protect the airdrop. The compromised key stays the only publisher and may publish a list naming itself, which the vault allows by design.
- Nothing in chain state explains the missing campaign. Holders only see the burn.

RECOMMENDATION

Give the vault a way to move `creator` with the launch. Two options, either is enough:

1. Add a creator transfer on the vault, callable only by the current `creator`:

```
function transferCreator(address newCreator) external onlyCreator {
    if (newCreator == address(0)) revert ZeroAddress();
    emit CreatorTransferred(creator, newCreator);
    creator = newCreator;
}
```

2. Or make `updateCreator` rebind the vault too: look up `airdropVaultOf(poolId)` on the factory and call a hook-only setter on the vault in the same transaction, so the two can never drift.

Until then, the front-end should warn before `updateCreator` whenever `airdropVaultOf(poolId)` holds a vault with an unfinished campaign.

08 POST-AUDIT STATEMENT

Post-Audit Statement

This engagement surfaced one finding, of Low severity, which was resolved at the remediation commit.

SCOPE OF THIS STATEMENT

The review covered CashCatAirdropVault.sol at the review commit and its integration points in CashCatFactoryVNext and CashCatHookV2 as far as they touch the vault. The remediation review covered only the diff that closes the finding. Other changes in the same commit range, and the rest of the letscash codebase, are covered by their own reports.

AUDIT COMPLETE

Stay secure.

Thank you for choosing SBSecurity.

WEBSITE
sbsecurity.net

TELEGRAM
@blckhv

REPORT
letscash · August 2026